

Case Study 1

The idea of this case study is to familiarize you with Gradio for building user interfaces, Hugging Face for hosting machine learning models, and Git for version control and collaboration. You will then experiment with deploying machine learning applications to compare two different approaches:

1. running a model on the same host that is running your product (e.g., using the compute resources provided by the Hugging Face Space, without making a remote API call)
2. using a remote API to access a hosted LLM (e.g. using the InferenceClient function from the `huggingface_hub` library)

The remotely hosted approach should use an LLM, while the locally executed approach may use either an LLM or another appropriate machine learning model.

Additionally, imagine an evil CEO who does not trust Hugging Face as a code repository and insists that all code be stored on GitHub instead. To accommodate this, you will clone your Hugging Face repository to GitHub and set up a GitHub Action that automatically synchronizes the two repositories, ensuring that any changes made on GitHub are reflected in the repository on Hugging Face Spaces. The evil CEO also wants to make sure everything is working properly—you will also experiment with using GitHub Actions for hosting or testing your model.

The project will be completed in groups of 1-3 people each. The total number of points for the case study will be 100 (with 10 points of possible extra credit), distributed as shown below.

You are allowed to base your products on existing Hugging Face Spaces projects, but you are encouraged to make them unique—experiment with Gradio programming, change the interface, add buttons, sliders, or any other elements to enhance the user experience.

Also, you must cite any Hugging Face Spaces, LLM assistance, and any other support you get for the project.

Deliverables

This case study includes 5 deliverables:

1. An LLM-based product that uses a remotely hosted model through an API and is hosted on Hugging Face Spaces **(25 points)**
 - a. The source code for the API-based LLM product *(15 points)*

As a starting point, you are welcome to use a default Hugging Face Space project or copy [the class example](#), which has the functionality of using both an API-based model and a locally hosted one. While a complex product is not required, we do expect some additions if you do start from an already existing space.
 - b. Copy of a GitHub Action that automatically syncs the repository with Hugging Face Spaces whenever there is a commit or push to the GitHub repository *(10 points)*

Please follow the example from [Managing Spaces with GitHub Actions](#) and ensure that you correctly configure the necessary authentication tokens and repository settings. Be sure to watch out for any branch specifications, required environment variables, and the correct setup of repository secrets (like the Hugging Face token).
 - c. Please provide links to your Hugging Face and GitHub repositories for grading.
2. A product that runs the model locally on the same host as the application, using either the compute resources provided by Hugging Face Spaces or your own machine **(25 points)**
 - a. The source code for the locally executed product *(15 points)*

Feel free to look through [the class example](#) for an example of locally running a model through a Transformers Pipeline. Like the API-based product, using these examples as a starting point is allowed, but make the product your own through personalized additions.
 - b. Copy of a GitHub Action that automates testing or running the model locally on Hugging Face Spaces *(10 points)*

For example, you can create a unit test (e.g., using 'pytest') to check your model's functionality and add a GitHub Action workflow to run these tests automatically on every commit or pull request. Your workflow should include steps to install dependencies, set up the environment, and run the tests. Feel free to refer to the [GitHub Actions documentation](#) for inspiration.
 - c. Please provide links to your Hugging Face and GitHub repositories for grading.

3. Messaging your team when the GitHub action completes (**10 points**)

Using something like [Intro to Webhooks – Discord](#), have the GitHub action automatically message your team whenever the action runs. Other message systems (such as Slack) have similar capabilities. Feel free to use whatever messaging system you like.

- a. Submit a link to your GitHub repository for grading with a description of what you implemented.

4. A report for the project—1-2 pages long (**20 points**)

The report should include the following. Unless specifically marked as **[LLM only]** or **[Human only]**, you may use LLM assistance as you see fit, but all LLM usage must be cited:

- a. Members' names (1 point)
- b. Description of the product(s), their purpose, and target audience (2 points)
- c. Description of the models being used, including their architecture, purpose, and any datasets involved (3 points)
- d. Performance analysis of the products (e.g., response time, resource usage, and any noticeable differences between the remotely hosted and locally executed models) (2 points)
- e. Cost-based analysis of the products (e.g., estimated cost if you have 1,000 users) (2 points)
 - i. See Hugging Face – Pricing for details on the costs associated with hosting and using models on Hugging Face Spaces.
 - ii. Remember that rate limits and usage limits may apply to hosted inference, so consider these limits when estimating costs for a large number of users.
- f. Comments and/or concerns, such as potential security issues, data privacy, scalability, reliability, and dependence on an external API (2 points)
- g. **[LLM only]** Use an established LLM such as ChatGPT, Claude, Gemini, or another comparable model to analyze your code and product. Ask it to identify one meaningful improvement related to performance, reliability, scalability, cost, security, or maintainability. Include the complete prompt, the LLM/model used, and its response. (2 points)
- h. **[LLM only]** Ask your locally executed or otherwise weaker LLM the same question using the same prompt. Include the model used and its response. (2 points)
- i. **[Human only]** Compare the two LLM responses. Which response was more useful, accurate, and actionable? Identify any incorrect, vague, or

questionable recommendations, and explain which suggestions you would actually consider implementing and why. (2 points)

- j. **[Human only]** Based on your results, if you were deploying this product for real users, which model would you choose and why? Would your answer change if cost, privacy, or reliability were the highest priority? (2 points)
 - k. Submit your report on canvas.wpi.edu for grading.
5. A 2–3-minute video pitch to a Venture Capitalist **(20 points)**
- This should be a demonstration of your product, including its features, user experience, and benefits. This is the one product-focused part of the assignment; highlight what makes your product unique and valuable. Explain how it addresses a real-world problem, the potential market impact, and why it stands out compared to existing products. You should also incorporate any findings from your report that would be relevant to a potential investor, such as performance, cost, scalability, reliability, or other trade-offs that affect the viability of the product.
- a. Submit your video on canvas.wpi.edu for grading.

Extra Credit:

6. Adaptive LLM Exception Handling/Failover **(10 points, extra credit)**
- Extend your application so that it can automatically switch between the remotely hosted LLM and the locally executed LLM when necessary. For example, your application may switch to the locally executed model if the remote API is unavailable, times out, or reaches a usage or rate limit. Similarly, the application may switch back to the remotely hosted model when it becomes available again. Your implementation should:
- a. Automatically detect at least one type of failure or unavailable model.
 - b. Automatically route the request to the alternative LLM without requiring the user to manually select it.
 - c. Clearly indicate which model handled the request.
 - d. Demonstrate and document the failover behavior.
 - e. Include a short discussion of the advantages and disadvantages of this approach.

Submit a link to your GitHub repository for grading with a description of what you implemented.

All materials should be submitted on canvas.wpi.edu.

Links to materials that will help with the case study:

- [Spaces \(huggingface.co\)](https://huggingface.co/spaces)
- [Gradio Spaces \(huggingface.co\)](https://huggingface.co/spaces/Gradio)
- [Managing Spaces with Github Actions \(huggingface.co\)](https://huggingface.co/spaces/Gradio)

Below is an example of the default Chatbot Interface as opposed to a Gradio project that has modified the default layout and added functionality:

Default:



After:

